

Narzędzia programistyczne - kompilator, linker, debugger, edytor tekstowy, platforma programistyczna, zintegrowane środowisko programistyczne

Uniwersytet Morski w Gdyni
Narzędzia Informatyczne

Na początek ...

Jest wiele osób, które chcą nauczyć się programowania.

Jest wiele osób, które próbują to robić.

Jest wiele osób, które zniechęciły się zanim zaczęły programować.

Narzędzia programistyczne

- Tworzenie aplikacji (edytory, kompilatory, interpretatory)
- Wykrywanie błędów: statyczna/dynamiczna analiza kodu/programu
- „Odpluskwianie” kodu (Debugging)
- Analiza wydajności aplikacji (profilowanie kodu)
- Analiza pamięci - wykrywanie „wycieków”
- Zarządzanie projektami, np.: systemy kontroli wersji, raportowanie błędów
- Tworzenie dokumentacji

Generacje języków programowania

- Pierwsza generacja — kod maszynowy
- Druga generacja — asemblery
- Trzecia generacja — języki wysokiego poziomu
 - ★ języki imperatywne (proceduralne)
 - ★ języki aplikatywne (funkcjonalne)
 - ★ języki deklaratywne (regułowe)
 - ★ języki symboliczne
 - ★ języki obiektowe

Generacja trzy i pół — środowiska programistyczne

- Czwarta generacja — języki idealnie nieproceduralne, generatory aplikacji
- Piąta generacja — inteligentne systemy wiedzy

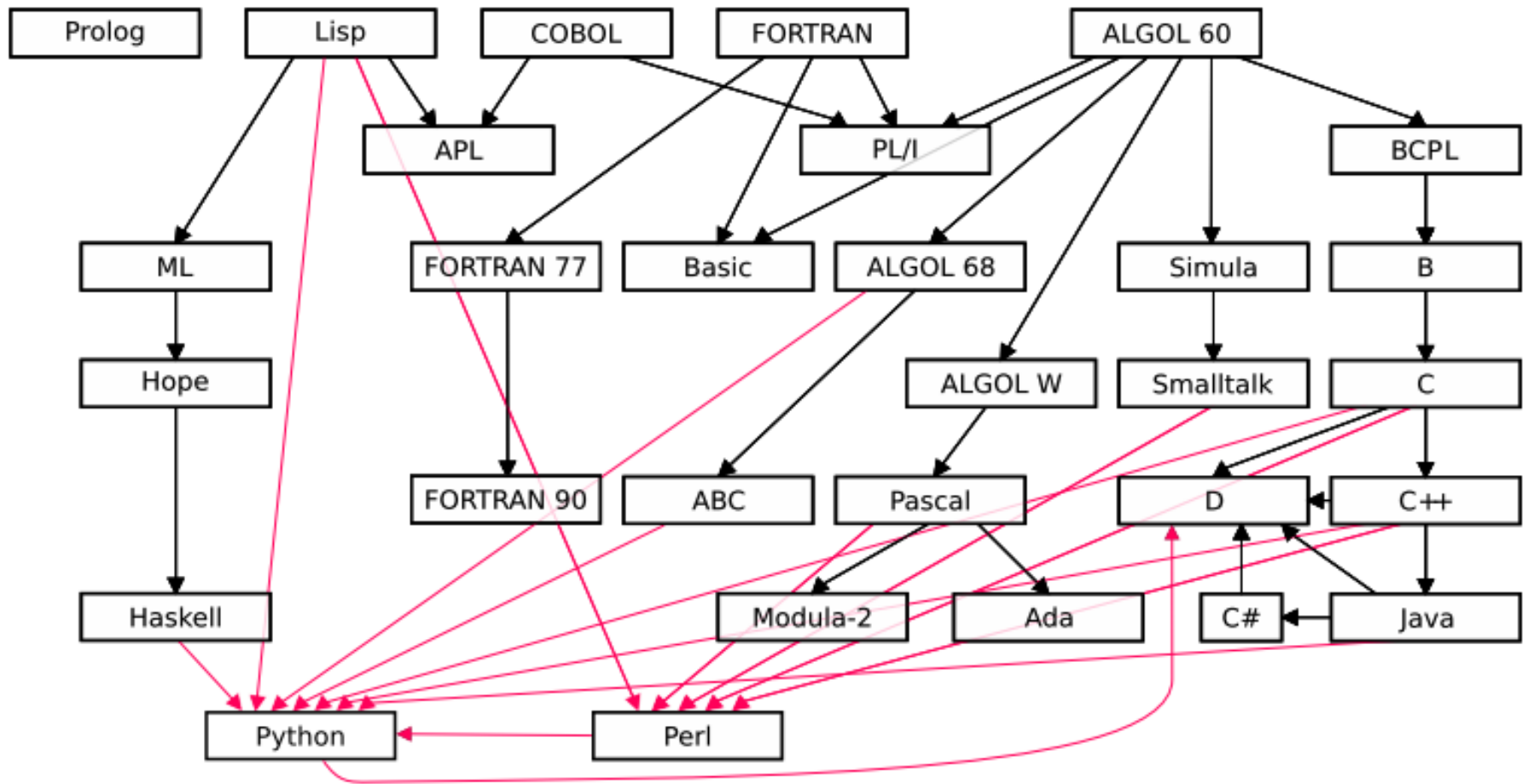
Przykłady języków wysokiego poziomu

- ★ języki imperatywne (proceduralne) — Fortran, ALGOL, COBOL, C
- ★ języki aplikatywne (funkcjonalne) — Lisp, Scheme, Haskell
- ★ języki deklaratywne (regułowe) — Prolog, CLIPS, SQL
- ★ języki symboliczne — Lisp, Prolog, Mathematica, Maple
- ★ języki obiektowe — Smalltalk, Ada95, Lisp, C++, Java

Środowisko programistyczne

Język programowania wysokiego poziomu + wbudowane funkcje dla systemów informacyjnych (obsługa ekranu, bazy danych itp.) + interfejs graficzny — buildery i wizardy

Drzewo genealogiczne języków wysokiego poziomu (domniemane)



Co więc wybrać

Wyjątki z listy 50 najbardziej popularnych języków programowania (www.tiobe.com/tiobe-index/):

- 16 Asembler (1949)
- 39 Fortran (1957)
- 27 Lisp (1958)
- 13 BASIC (1964) (Visual Basic .NET (2001) 6 miejsce)
- 37 Prolog (1972)
- 2 C (1972) (15% rynku, przyrost 8,09%)
- 3 C++ (1985) (8% rynku, przyrost 2,61%)
- 14 Perl (1987)
- 4 Python (1990)
- 1 Java (1995) (15% rynku, przyrost 0,88%)

Wniosek:

Nowe języki powstają ustawicznie i koegzystują ze starszymi

Jak wybrać

Obserwacja:

Typowy programista, inżynier oprogramowania (architekt:) z kilkuletnim doświadczeniem programuje w kilkunastu językach

Zamiast pytać:

Jaki język programowania jest obecnie najpopularniejszy?

warto zapytać:

Który język programowania jest obecnie niedoceniany i pozwala robić rzeczy, których inni programiści nie są w stanie zrobić, tym samym dając mi unikalne umiejętności?

Popularne kryteria wyboru języka programowania do nauki

- Wysokość uposażenia
- Popularność – ogrom różnorodnych ofert pracy
- Wzrostowa tendencja popularności
- Łatwość nauczania, komfort pracy

Języki, które „dobrze rokują”:

- Python
- Perl
- C#
- Ruby
- Rust
- Elixir
- Scala
- R
- plus biblioteki!

Warto zapamiętać także, że:

Tworzone aplikacje działają w jakimś środowisku sprzętowym, zazwyczaj pod nadzorem jakiegoś systemu operacyjnego, więc...

Popularność według dziedzin

- oprogramowanie niskiego poziomu, sprzętowe, jądro systemu: **C, assembler**
- oprogramowanie zorientowane sprzętowo: **C/C++, assembler, Lua, Java**
- automatyka przemysłowa: **PLC** (Siemens, Allen-Bradley, Mitsubishi Electric, Beckhoff), **SCADA, DCS** → **Industrial IoT** (**Python, Java, C/C++** z protokołami komunikacji), **MATLAB, LabVIEW** (język **G**)
- robotyka: **C/C++, Python, Java, C#.NET, Matlab, assembler, HDLs** (FPGA), **Lisp**, języki programowania robotów – **RAPID** (ABB), **KRL** (Kuka), **Karel** (Fanuc), **AS** (Kawasaki), **INFORM** (Yaskawa), **VAL3** (Staubli), **URScript** (Universal Robots), **ROS Industrial**
- aplikacje www: np. **Java, Python, Ruby** (wraz z platformami takimi jak **Django, Ruby on Rails** itp.)
- aplikacje mobilne: **HTML, Swift, C++, C#, Java**
- gry: **C/C++, Java, C#, Visual Basic .NET, Swift, Lua, Python, Lisp, Perl, Smalltalk** z dedykowanymi bibliotekami (**SDL, Allegro**), API i SDK
- chmury obliczeniowe: **SQL, XML, R, Clojure, Haskell, Scala, OCaml, F#, Common Lisp, Scheme**
- blockchain: **C++, Java, Python, Simplicity, Solidity, Serpent, LLL**
- roboty Boston Dynamics: **C, C++, Python z ROS**
- robot Sophia: **Java, C#, C++, OSGi, CogChar, Prolog, AIML, ChatScript, SingularityNET**

Kod maszynowy i asembler

Dodanie dwóch liczb na MC68HC705J1A (Motorola, 8-bitowy)

Kod maszynowy

Komorka Pamieci	Zawartosc
0300	A6
0301	02
0302	AB
0303	02
0304	CC
0305	03
0306	04
07FE	03
07FF	00

Asembler

```
org $0300
DODAJ  lda #2      ;zaladuj do akumulatora 2
      add #2      ;dodaj do akumulatora 2
PETLA  jmp PETLA   ;skocz do PETLA
      org $07FE
      dw  DODAJ    ;adres startu po resecie
```

Po przetworzeniu

0300	1	org	\$0300
0300 A602	2	DODAJ	lda #2
0302 AB02	3		add #2
0304 CC0304	4	PETLA	jmp PETLA
07FE	5		org \$07FE
07FE 0300	6		dw DODAJ

Symbol Table

PETLA	0304
DODAJ	0300

To samo w S-rekordach

```
S10C0300A602AB02CC0304C8
S10507FE0300F2
S9030000FC
```

Języki wysokiego poziomu

- język imperatywny — C

```
int dodaj()  
{ int a = 2;  
  int b = 2;  
  return(a + b);}
  
int l = dodaj();
```

- język aplikatywny — Lisp

```
(defun dodaj()          ;aplikatywnie  
  ((lambda (x y) (+ x y)) 2 2))  
(defun dodaj()          ;imperatywnie  
  (let (x y)(setq x 2)(setq y 2)(+ x y)))  
  
(dodaj)
```

- język deklaratywny — Prolog

```
dodaj(X) :- X is 2 + 2.  
  
dodaj(X).
```

- język symboliczny — Mathematica

```
dodaj := Module[{a = 2, b = 2}, a + b];  
l = dodaj;
```

- język obiektowy — C++

```
class CDodaj  
{ public:  
  int a;  
  int b;  
  CDodaj()  
  { a = 2;  
    b = 2;};  
  int Dodaj()  
  { return(a + b);}  
};  
  
/*statycznie*/  
Cdodaj d;  
int l = d.Dodaj();  
  
/*dynamicznie*/  
Cdodaj *d = new CDodaj;  
int l = d->Dodaj();
```

Języki czwartej generacji

Elementy składowe języka czwartej generacji:

- fizyczny słownik danych
- formater ekranu
- generator raportów
- język zapytań
- specyfikator dialogu
- generator kodu
- język wysokiego poziomu

Języki czwartej generacji

Języki czwartej generacji:

- CASE — Computer Aided Software Engineering
- Oracle — Komercyjny pakiet zawierający:
 - ★ maszynę relacyjnej bazy danych;
 - ★ interfejs bazy danych
 - ★ narzędzia do integracji baz danych z językami wysokiego poziomu;
 - ★ kreator formularzy;
 - ★ kreator raportów;
 - ★ kreator reprezentacji graficznych;
 - ★ narzędzia do wyszukiwania
- UML — Unified Modeling Language

Symulacja systemu robotycznego cd.

- C++

```
class CRobot
{ private:
    int id
  public:
    char      Nazwa[42];
    CStan     Stan;
    CPrzegub  *Przeguby;
    CRezolwer *Rezolwery;
    CKolejka  *Kolejka = NULL;

    CRobot()
    { Przeguby=new CPrzegub[5]; /*...*/
      Inicjuj(); /*...*/};

    int KolejkaZadan(){/* definicja */};
    int DodajZadanie(/*...*/){/* def. */};
    int UsunZadanie(/*...*/){/* def. */};};

    CRobot *Robot1 = new CRobot;
    try{Robot1.DodajZadanie(/*...*/);}
    catch(CException blad){/* obsluga */}
```

- C

```
struct SRobot
{ int id
  char      Nazwa[42];
  SStan     Stan;
  SPrzegub  *Przeguby;
  SRezolwer *Rezolwery;
  SKolejka  *Kolejka = NULL;};

int DodajRobota(SRobot *Robot)
{ Robot = malloc(sizeof(SRobot));
  Robot->Przeguby=malloc(5*sizeof(SPrzegub));
  Inicjuj(); /*...*/};

int UsunRobota(SRobot *Robot){/*...*/};
int KolejkaZadan(/*...*/){/* def. */};
int DodajZadanie(/*...*/){/* def. */};
int UsunZadanie(/*...*/){/* def. */};};

SRobot *Robot1;
DodajRobota(Robot1);
if blad = DodajZadanie(/*...*/) then
  /* obsluga */}
```

Symulacja systemu robotycznego cd.

- Asembler i kod maszynowy

98: float CRobot::UstalKrok(float Krok, int Kp)

99: {

```
00402828  push      ebp
00402829  mov       ebp,esp
0040282B  sub       esp,0Ch
0040282E  mov       dword ptr [ebp-8],ecx
```

100: float delta = 0.001;

```
00402831  mov       dword ptr [delta],3A83126Fh
```

101: switch(Kp){

```
00402838  mov       eax,dword ptr [Kp]
0040283B  mov       dword ptr [ebp-0Ch],eax
0040283E  cmp       dword ptr [ebp-0Ch],0
00402842  je        CRobot::UstalKrok(0x0040284c)+24h
00402844  cmp       dword ptr [ebp-0Ch],1
00402848  je        CRobot::UstalKrok(0x00402857)+2Fh
0040284A  jmp       CRobot::UstalKrok(0x00402862)+3Ah
```

102: case 0: Krok += delta;

```
0040284C  fld       dword ptr [Krok]
0040284F  fadd      dword ptr [delta]
00402852  fstp      dword ptr [Krok]
```

103: break;

```
00402855  jmp       CRobot::UstalKrok(0x00402869)+41h
```

104: case 1: Krok -= delta;

```
00402857  fld       dword ptr [Krok] ...
```


Języki symboliczne — Mathematica

```
Sqrt[4]
```

```
2
```

```
Sqrt[-4]
```

```
2i
```

```
Sqrt[a]
```

```
 $\sqrt{a}$ 
```

```
Sqrt[a]^2 + a + 3b + 5b
```

```
2a + 8b
```

```
Solve[x^2 + 5x + 4 == 0, x]
```

```
{{x → -4}, {x → -1}}
```

```
Solve[ax^2 + bx + c == 0, x]
```

```
{{x →  $\frac{-b - \sqrt{b^2 - 4ac}}{2a}$ }, {x →  $\frac{-b + \sqrt{b^2 - 4ac}}{2a}$ }}
```

```
Solve[Sqrt[x] + a == 2x, x]
```

```
{{x →  $\frac{1}{8}(1 + 4a - \sqrt{1 + 8a})$ }, {x →  $\frac{1}{8}(1 + 4a + \sqrt{1 + 8a})$ }}
```

```
Integrate[Sqrt[x] Sqrt[a + x], x]
```

```
 $\sqrt{a+x} \left( \frac{a\sqrt{x}}{4} + \frac{x^{3/2}}{2} \right) - \frac{1}{4}a^2 \log[\sqrt{x} + \sqrt{a+x}]$ 
```

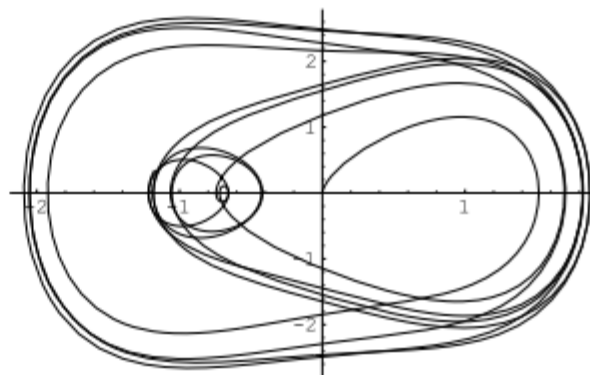
```
 $\int \sqrt{x} \sqrt{a+x} dx$ 
```

```
 $\sqrt{a+x} \left( \frac{a\sqrt{x}}{4} + \frac{x^{3/2}}{2} \right) - \frac{1}{4}a^2 \log[\sqrt{x} + \sqrt{a+x}]$ 
```

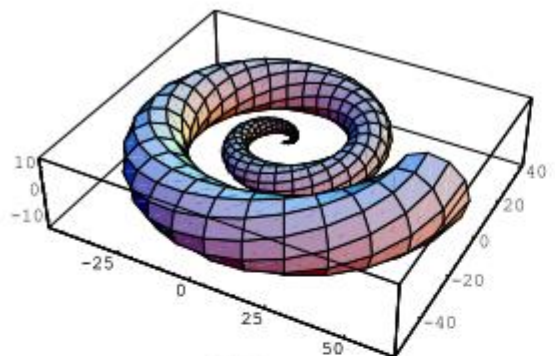
```
NDSolve[{x''[t] + x[t]^3 == sin[t], x[0] == x'[0] == 0},  
x, {t, 0, 50}]
```

```
{{x → InterpolatingFunction[{{0., 50.}}, <>]}}
```

```
ParametricPlot[Evaluate[{x[t], x'[t]}/.%, {t, 0, 50}];
```

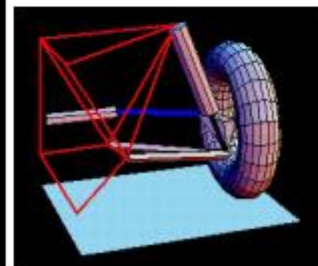
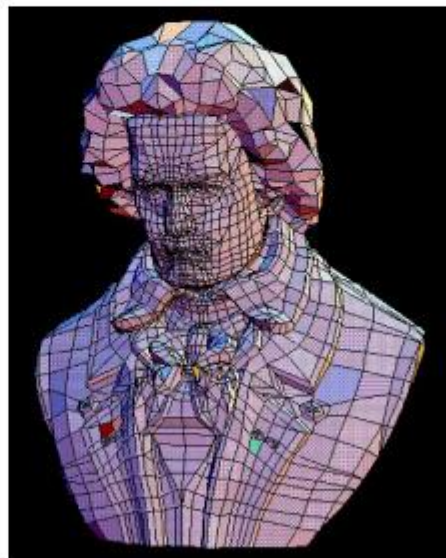
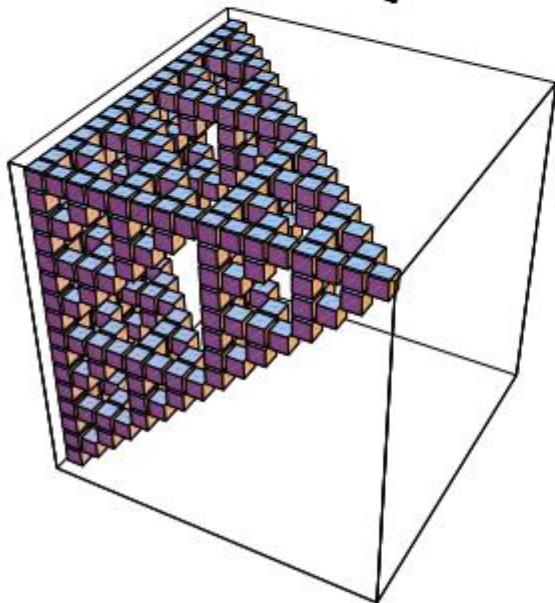


Języki symboliczne — Mathematica



```
ParametricPlot3D[{u Cos[u](4 + Cos[v + u]), u Sin[u](4 + Cos[v + u]), u Sin[v + u]}, {u, 0, 4 Pi}, {v, 0, 2 Pi}, PlotPoints -> {60, 12}];
```

```
Show[Graphics3D[Flatten[Table[If[Mod[Multinomial[x, y, z], 2] == 1, Cuboid[1.2 {x, y, -z}], {}], {x, 0, 15}, {y, 0, 15}, {z, 0, 15}]]]]]
```



Inne języki

- języki opisu strony — `html`, `TEX/LATEX`
- języki manipulowania tekstem — `sed`, `awk`
- meta-języki — `Lex/Flex`, `Yacc/Bison`
- APL — A Programming Language

```
▽ FIB N  
[1] A←1 1  
[2] →2×N>⍴A← A, +/−2↑A▽
```

Założenia paradygmatu obiektowego

- **Programowanie obiektowe** (ang. *object-oriented programming*) to:
paradygmat programowania, w którym programy definiuje się za pomocą **obiektów** — elementów łączących **stan** (czyli **dane**, nazywane najczęściej polami) i **zachowanie** (czyli **metody** które są procedurami/funkcjami operującymi na tych danych).
- Obiektowy program komputerowy wyrażony jest jako **zbiór takich obiektów**, komunikujących się pomiędzy sobą w celu wykonywania zadań.
- Podejście to różni się od tradycyjnego **programowania proceduralnego**, gdzie dane i procedury nie są ze sobą bezpośrednio związane.
- Programowanie obiektowe ma ułatwić pisanie, konserwację i wielokrotne użycie programów lub ich fragmentów.

Założenia paradygmatu obiektowego

- **Obiekt** (zmienna obiektowa) jest tzw. **INSTANCJĄ** klasy. Jest częścią programu i wykonuje ściśle określone zadania.

Programowanie oparte na klasach

- **Klasa** to typ obiektowy składający się z pól i metod. Definiowane są klasy, czyli typy zmiennych, a następnie tworzone są obiekty, czyli zmienne (w uproszczeniu) tych typów.
- **Pole** to zmienna która należy do klasy. Wartości pól należy modyfikować przez metody danej klasy.
- **Metoda** to procedura lub funkcja która należy do klasy i wykonuje operacje na wartościach pól.

Założenia paradygmatu obiektowego – c.d.

Powszechnie uważa się, że język programowania jest obiektowy jeżeli posiada następujące cechy:

1. Hermetyzacja/Enkapsulacja

To cecha polegająca na ukrywaniu szczegółów implementacji. Zapewnia, że obiekt nie może zmieniać stanu wewnętrznego innych obiektów w nieoczekiwany sposób. Tylko wewnętrzne metody obiektu są uprawnione do zmiany jego stanu.

Definiując klasę stosuje się różne **poziomy dostępu**:

public

private

protected

Założenia paradygmatu obiektowego – c.d.

2. Polimorfizm /Wielopostaciowość

Mechanizm zezwalający na nadawanie w hierarchii klas tych samych nazw metodom o podobnym zastosowaniu.

Wywołanie metody spowoduje zachowanie odpowiednie dla typu obiektu wywoływanego. Jeśli dzieje się to w czasie działania programu, to nazywa się to *późnym wiązaniem* lub *wiązaniem dynamicznym*.

Relacje między klasami

- Relacja „jest”, zwana również dziedziczeniem: obiekt jednej klasy należy również w pewnym sensie do innej, bardziej podstawowej klasy. Np. każdy obiekt należący do klasy „Student” jest również obiektem klasy „Osoba”.
- Relacja „ma”: obiekt jednej klasy posiada obiekt innej klasy jako jedną ze swoich składowych. Np. Obiekt „Portfel” posiadać może jako jedną ze swoich składowych obiekt „Obligacja”.

Założenia paradygmatu obiektowego – c.d.

3. Dziedziczenie

Dziedziczenie to mechanizm budowania klas na podstawie klas już zdefiniowanych.

Klasa pochodna dziedziczy **składowe** od przodka –
klasy bazowej.

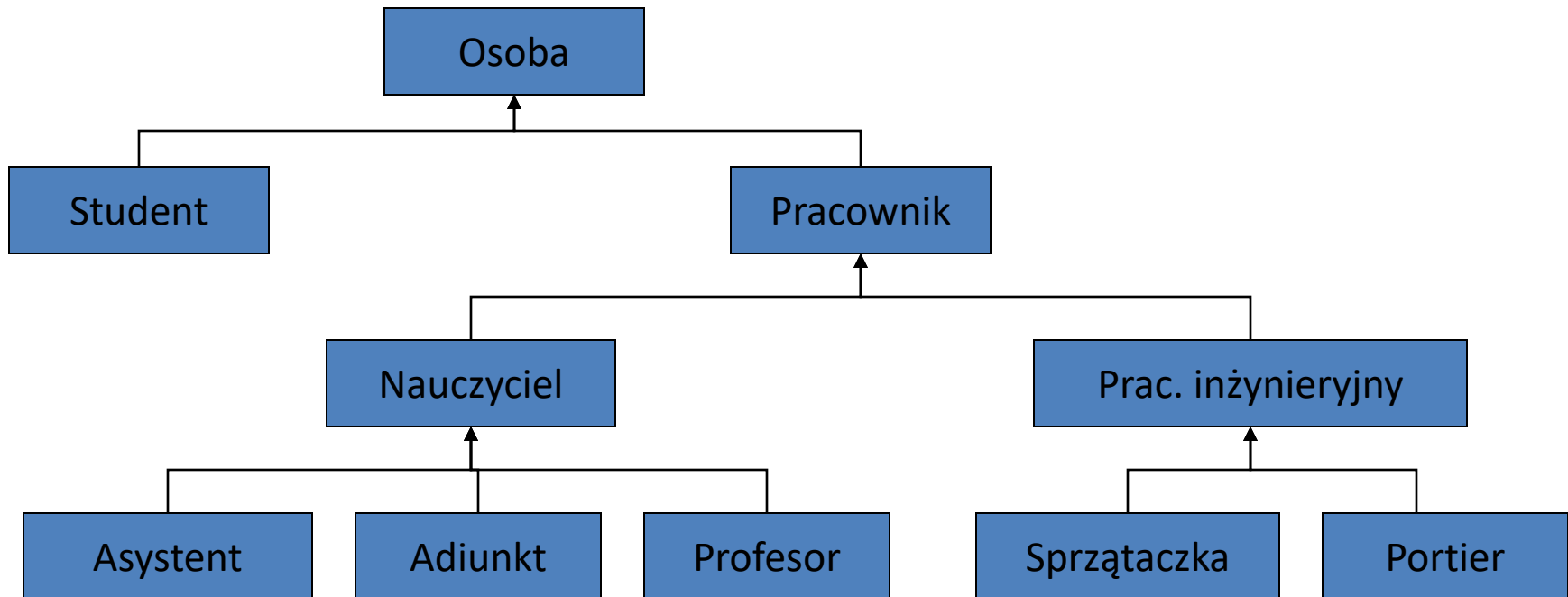
Dziedziczenie porządkuje i wspomaga polimorfizm i enkapsulację dzięki umożliwieniu definiowania i tworzenia specjalizowanych obiektów na podstawie obiektów bardziej ogólnych.

Dziedziczenie („jest”)

- Dziedziczenie dotyczy w sytuacji w której obiekty jednej klasy mają wszystkie cechy obiektów drugiej klasy jak również ewentualne własne cechy dodatkowe. Np. każdy obiekt klasy „Student” ma wszystkie cechy obiektu klasy „Osoba” oraz pewne dodatkowe atrybuty i metody. Mówimy wówczas, że klasa „Student” dziedziczy wszystkie właściwości klasy „Osoba”.
- Dziedziczenie odzwierciedla związek generalizacji, między pojęciem (klasą) ogólniejszą (taką jak „Osoba”) i pojęciem (klasą) bardziej szczegółową (taką jak „Student”).
- Klasę ogólniejszą nazywamy klasą podstawową (lub bazową), klasę która z niej dziedziczy - klasą pochodną.

Hierarchie dziedziczenia

- Pojęcia układają się w pewne naturalne hierarchie generalizacji / uszczegółowiania. Struktury te mają przy tym charakter drzewiasty, tzn. jedno pojęcie (klasa) ogólniejsze może służyć jako podstawa większej liczby pojęć bardziej szczegółowych.



Założenia paradygmatu obiektowego – c.d.

4. Abstrakcja

Abstrakcją w programowaniu nazywamy pewnego rodzaju uproszczenie rozpatrywanego problemu, polegające na ograniczeniu zakresu cech obiektów wyłącznie do **cech kluczowych dla algorytmu**.

Każdy obiekt w systemie służy jako model abstrakcyjnego „wykonawcy”, który może wykonywać pracę, opisywać i zmieniać swój stan oraz komunikować się z innymi obiektami w systemie, bez ujawniania, w jaki sposób zaimplementowano dane cechy.

```
class TBankAccount {  
    // szczegóły pominięte  
    float _balance;  
    float _interestYTD;  
    TClient* _owner;  
    int    _accountNumber  
};
```

```
struct TBankAccount {  
    //szczegóły pominięte  
    float balance;  
    float interestYTD;  
    struct TClient*owner;  
    int    accountNumber  
};
```

```
class TBankAccount {  
    //many details omitted  
    float _balance;  
    float _interestYTD;  
    TClient* _owner;  
    int    _accountNumber  
};
```

```
struct TBankAccount {  
    //many details omitted  
    float balance;  
    float interestYTD;  
    struct TClient* owner;  
    int    accountNumber  
};  
// całe mnóstwo kodu  
  
int MakeDeposit(  
    long acctNo,  
    float amount);  
float Withdraw(  
    long acctNo,  
    float amount);  
float Print(long acctNo);
```

```
class TBankAccount {  
    // szczegóły pominięte  
    float _balance;  
    float _interestYTD;  
    TClient* _owner;  
    int    _accountNumber  
  
public:  
  
    void MakeDeposit(  
        float amount);  
    float Withdraw(  
        float amount);  
    void Print();  
};
```

```
struct TBankAccount {  
    // szczegóły pominięte  
    float balance;  
    float interestYTD;  
    struct TClient* owner;  
    int    accountNumber  
};  
// całe mnóstwo kodu  
  
void MakeDeposit(  
    long acctNo,  
    float amount);  
float Withdraw(  
    long acctNo,  
    float amount);  
void Print(long acctNo);
```

Dokumentowanie algorytmów

Pseudokodem

Pseudokodem nazywany jest taki sposób zapisu algorytmu, który zachowując strukturę charakterystyczną dla kodu zapisanego w języku programowania, rezygnuje ze ścisłych reguł składniowych na rzecz prostoty i czytelności.

Pseudokod nie zawiera szczegółów implementacyjnych (jak np. inicjalizacja zmiennych, alokacja pamięci itp.), często też opuszcza się w nim opis działania podprocedur (jeśli powinien być on oczywisty dla czytelnika), zaś nietrywialne kroki algorytmu opisywane są z pomocą formuł matematycznych lub zdań w języku naturalnym.

Nie istnieją w chwili obecnej szerzej przyjęte standardy zapisu pseudokodu. Większość autorów używa przyjętej *ad hoc* składni, często opierając się na składni istniejących języków programowania (Pascal, ALGOL, C).

Dokumentowanie algorytmów

Zastosowanie pseudokodu

Kod w **PHP**:

```
<?php
if (karta_wazna($nr_karty)) {
    wykonaj_transakcje($nr_karty, $zamowienie);
} else {
    informacja_o_bledzie();
}
?>
```

Pseudokod:

```
jeżeli numer karty kredytowej jest ważny to
    wykonanie transakcji w oparciu o numer karty i zamówienie
w przeciwnym razie
    wyświetlenie wiadomości o niepowodzeniu
koniec warunku
```

Program **Hello world** napisany w pseudokodzie ma prostą strukturę:

```
wyświetl: Hello World
```

Dokumentowanie algorytmów

Zastosowanie pseudokodu

Algorytm wczytuje i sumuje n liczb.

Start

Suma:=0

Podaj(n)

i:=0

Dopóki $i < n$ wykonuj:

 Wczytaj(a)

 Suma := Suma + a

$i := i + 1$

Wypisz(Suma)

Koniec

Algorytm wczytuje i dodaje 10 liczb.

Start

$i := 0$

Dopóki $i < 10$

 Wczytaj(a)

 Suma := Suma + a

$i := i + 1$

Koniec

Dokumentowanie algorytmów

Schemat blokowy

- Schematem blokowym nazywamy narzędzie służące do przedstawienia kolejnych czynności w projektowanym algorytmie. Jest to diagram, na którym procedura, system lub program komputerowy są reprezentowane przez opisane figury geometryczne połączone wektorami zgodnie z kolejnością wykonywania czynności wynikających z przyjętego algorytmu rozwiązania zadania.
- Schemat blokowy powinien zapewniać:
 - elastyczność zapisów
 - możliwość zapisu z użyciem składu wybranego języka programowania
 - łatwą kontrolę poprawności algorytmu.
- Schematy blokowe pozwalają na prostą zamianę instrukcji na instrukcje programu komputerowego.

Dokumentowanie algorytmów

Schemat blokowy

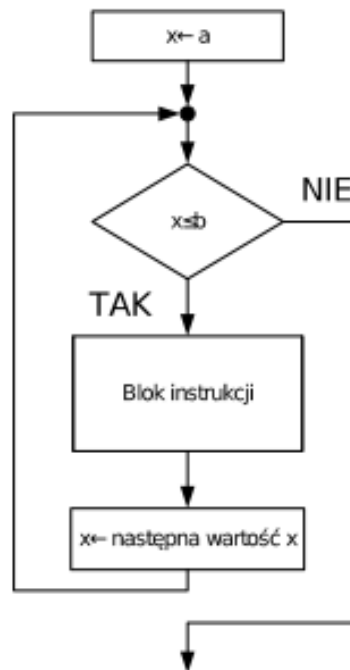
- Schemat blokowy to układ figur geometrycznych (nazywanych skrzynkami lub blokami) połączonych ze sobą odcinakami prostych lub łamanych (ścieżki sterujące).
- Figury służą do przedstawiania rodzaju działań zaprojektowanych w algorytmie, zaś linie wskazują kolejność wykonywania tych działań. Każda figura w schemacie blokowym reprezentuje określony rodzaj operacji.
- Zaletą schematów blokowych jest to, że graficznie prezentują algorytm zarówno od strony występujących w nim działań jak i ich kolejności.

Dokumentowanie algorytmów

Zastosowanie schematu blokowego

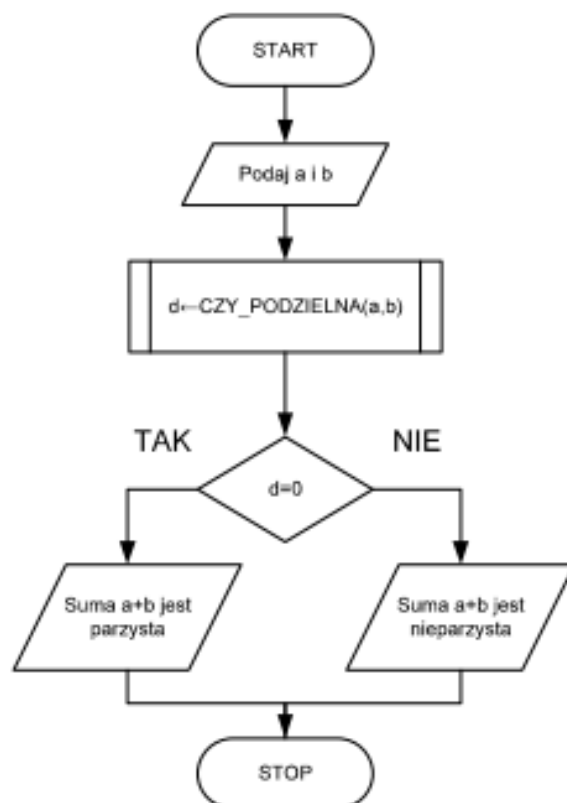
Pętla FOR

Schemat blokowy postaci

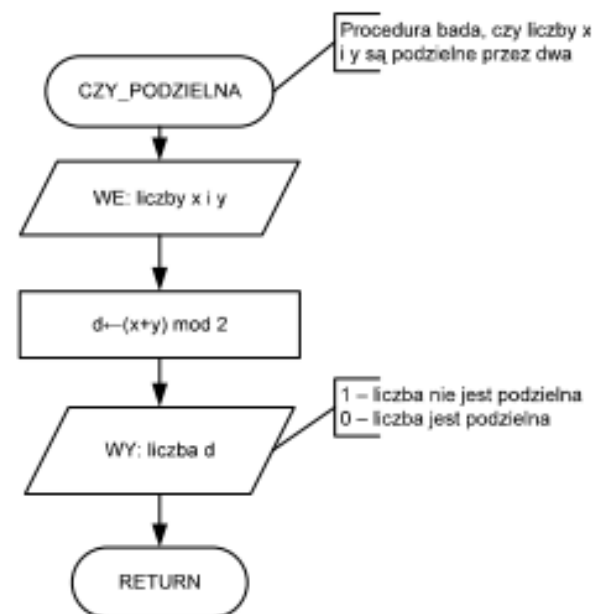


Dokumentowanie algorytmów

Zastosowanie schematu blokowego



(a) Program główny

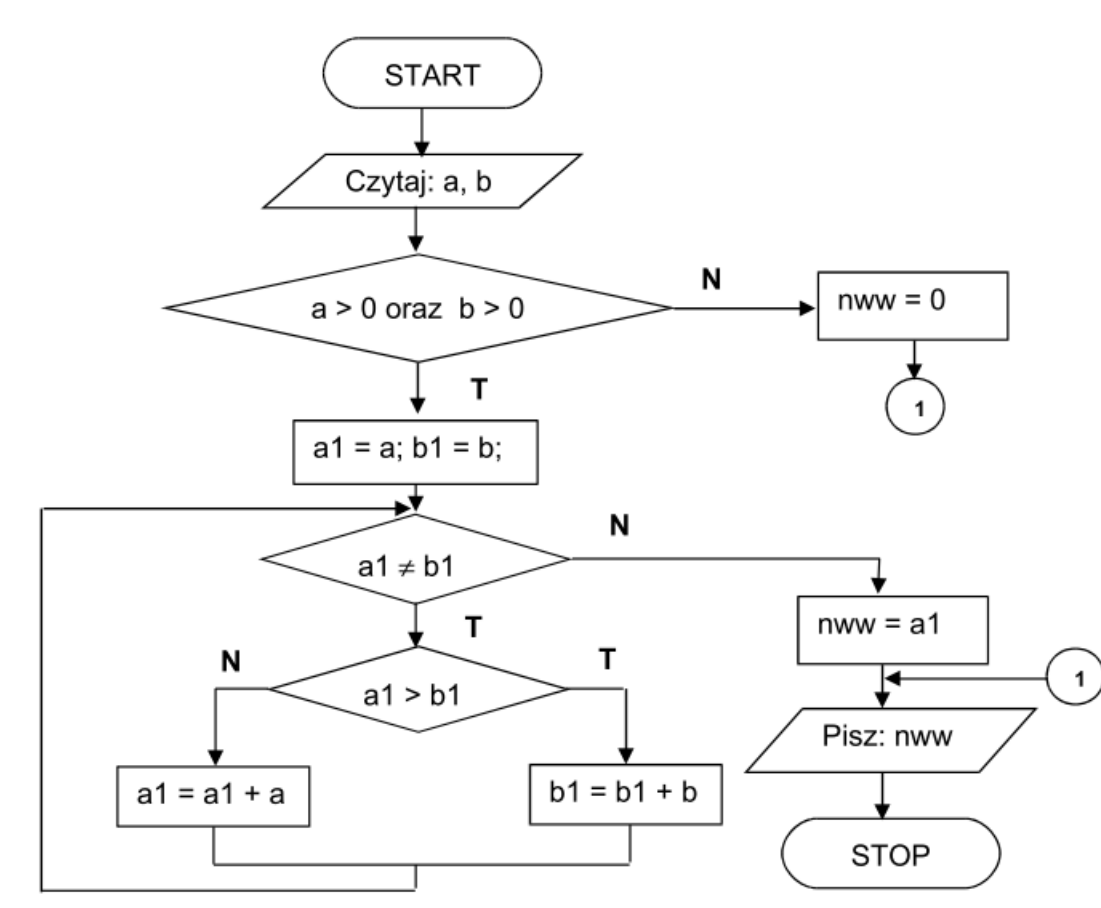


(b) Procedura

Dokumentowanie algorytmów

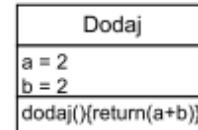
Zastosowanie schematu blokowego

Napisz program, który obliczy najmniejszą wspólną wielokrotność dwóch liczb całkowitych a i b dodatnich.



UML – Dokumentowanie

Dodanie dwóch liczb



Symulacja systemu robotycznego

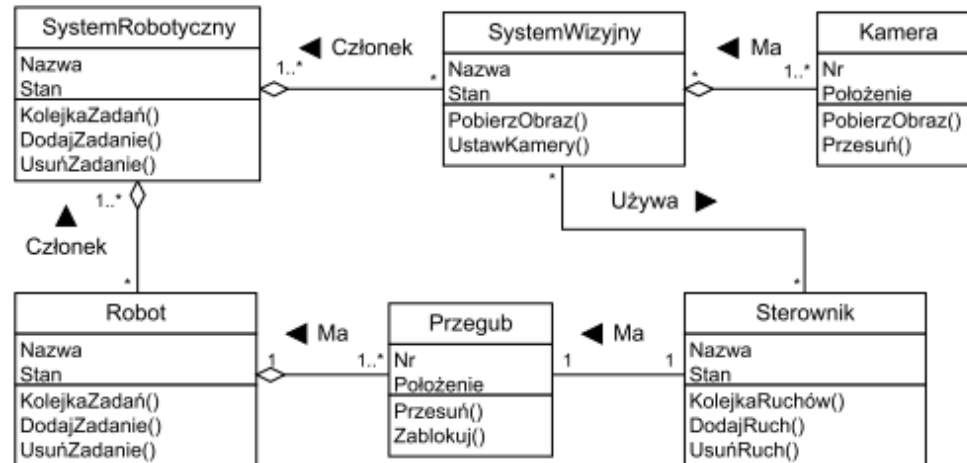
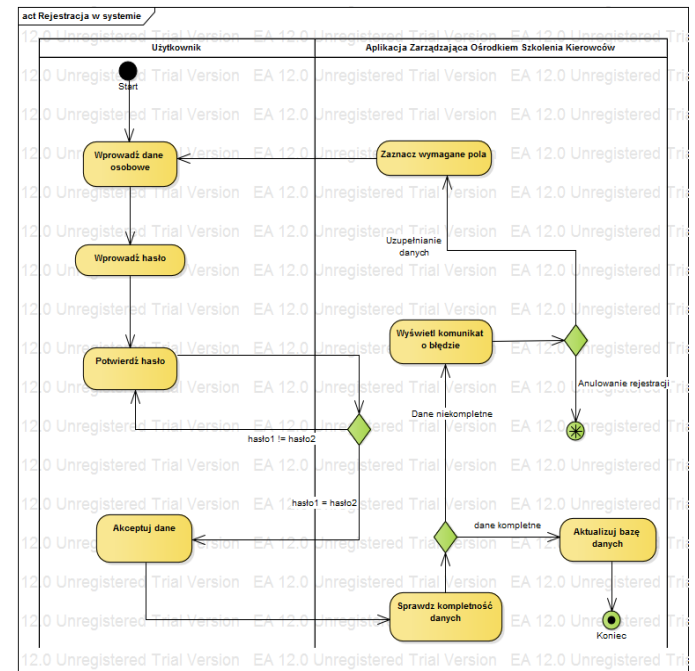
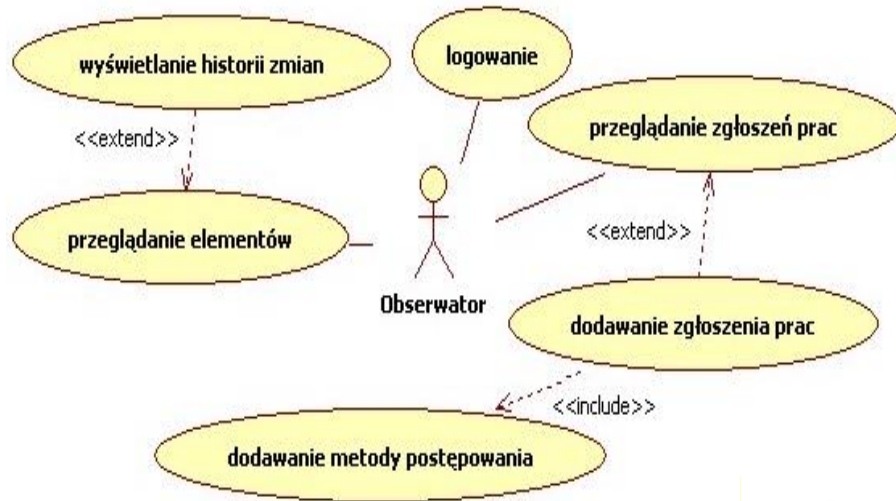


Diagram klas

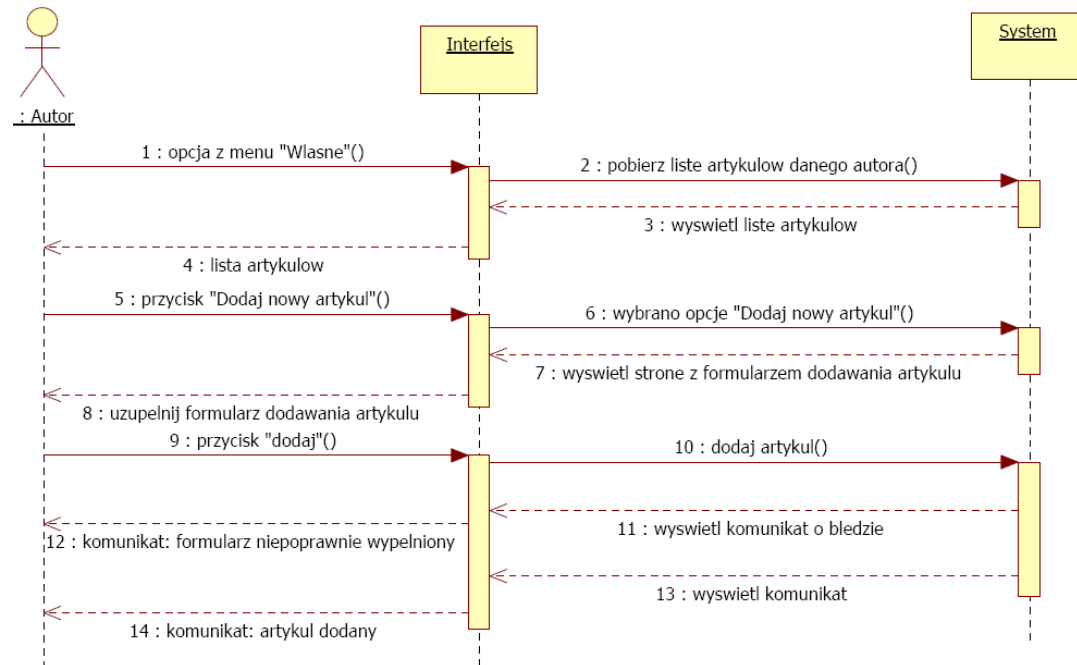
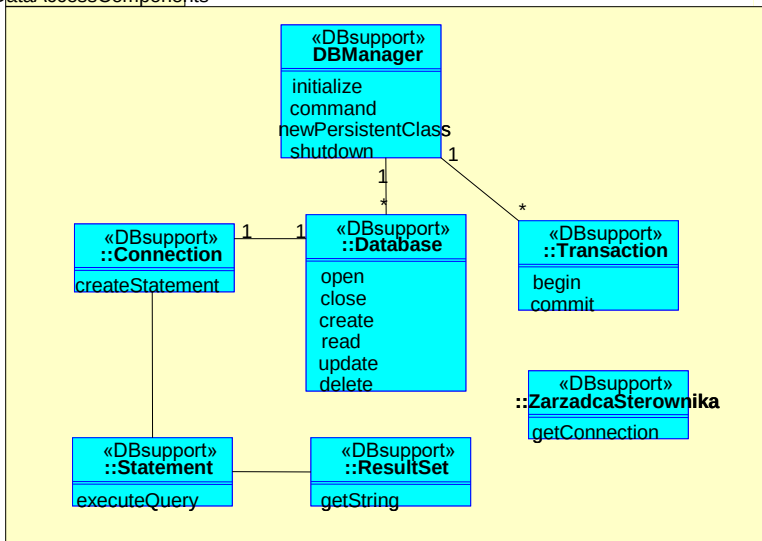
Do tego należy utworzyć:

- diagram obiektów
- diagram implementacji
- diagram rozmieszczenia
- diagram zachowań
- diagram sekwencji
- diagram interakcji

UML



::DataAccessComponents



Kompilator

Kompilator – program służący do automatycznego tłumaczenia kodu napisanego w jednym języku (języku *źródłowym*) na równoważny kod w innym języku (języku *wynikowym*). Proces ten nazywany jest kompilacją.

W informatyce kompilatorem nazywa się najczęściej program do tłumaczenia kodu źródłowego w języku programowania na język maszynowy. Niektóre z nich tłumaczą najpierw do języka asemblera, a ten na język maszynowy jest tłumaczony przez asembler.

Kompilatory mogą mieć możliwość automatycznej alokacji pamięci dla zmiennych, implementowania struktur kontrolnych lub procedur wejścia-wyjścia.

Stosowanie kompilatorów ułatwia programowanie (programista nie musi znać języka maszynowego) i pozwala na większą przenośność kodu pomiędzy platformami.

Kompilator

Log kompilacji

Log kompilacji zawiera informacje o tym czy proces kompilacji się powiódł. Jeśli podczas procesu kompilacji wystąpiły jakieś błędy to znajdą się one w logu wraz z ich opisem.

Kompilator często zamieszcza w logu miejsce wystąpienia błędu wskazując plik i wiersz w którym nie udało się tłumaczenie kodu.

Dodatkowo komunikat błędu zazwyczaj w miarę precyzyjnie 'mówi nam' co mu się nie spodobało podczas kompilacji.

Każdy komunikat jaki dostajesz w logu kompilacji powinieneś generalnie czytać i starać się go zrozumieć. Podczas nauki programowania bardzo często będziesz powtarzał te same błędy, więc nauczanie się czytania ich będzie dla Ciebie bardzo dużym ułatwieniem przy naprawianiu kodu.

Interpreter

Interpreter – program komputerowy, który analizuje kod źródłowy programu, a przeanalizowane fragmenty wykonuje.

Realizowane jest to w inny sposób niż w procesie kompilacji, podczas którego nie wykonuje się wejściowego programu (kodu źródłowego), lecz tłumaczy go do wykonywalnego kodu maszynowego lub kodu pośredniego, który jest następnie zapisywany do pliku w celu późniejszego wykonania.

Interpreter

Wykonanie programu za pomocą interpretera jest wolniejsze, a do tego zajmuje więcej zasobów systemowych niż wykonanie kodu skompilowanego, lecz może zająć relatywnie mniej czasu niż kompilacja i uruchomienie.

Jest to zwłaszcza ważne przy tworzeniu i testowaniu kodu, kiedy cykl edycja-interpretacja-debugowanie może często być znacznie krótszy niż cykl edycja-kompilacja-uruchomienie-debugowanie.

Interpreter

Interpretacja kodu programu jest wolniejsza od uruchamiania skompilowanego kodu, ponieważ interpreter musi najpierw przeanalizować każde wyrażenie i dopiero na tej podstawie wykonać odpowiednie akcje, a kod skompilowany wykonuje wyłącznie akcje.

W implementacjach będących w pełni interpreterami wielokrotne wykonanie tego samego fragmentu kodu wymaga wielokrotnej interpretacji tego samego tekstu. Ta analiza nazywana jest "kosztem interpretacji".

Dostęp do zmiennych jest także wolniejszy w przypadku interpretera, gdyż odwzorowanie identyfikatorów na miejsca w pamięci operacyjnej musi zostać dokonane podczas uruchomienia lub działania, a nie podczas kompilacji, dlatego niektóre interpretery tworzą dodatkowe dane (np. adresy zmiennych) przyspieszające wykonanie programu.

Języki interpretowane, kompilowane, skryptowe

Język interpretowany – język programowania, który zazwyczaj jest implementowany w formie interpretera. Program w języku interpretowanym nie jest kompilowany, lecz jest przechowywany w postaci kodu źródłowego i dopiero podczas uruchomienia wczytywany, interpretowany i wykonywany przez interpreter języka. Języki interpretowane ułatwiają pisanie programów przenośnych, czyli zgodnych z wieloma systemami operacyjnymi, np.: Bash, Perl lub Python.

Język kompilowany – język programowania, który by uzyskać działający program musi zostać wpierw skompilowany do postaci kodu maszynowego (inaczej: do postaci binarnej, w slangu komputerowym *binarka*). Najpopularniejszymi takimi językami są C++, C, Pascal.

Języki interpretowane, kompilowane, skryptowe

Język skryptowy – język programowania obsługujący skrypty. Często służący do kontrolowania określonej aplikacji.

Skrypty – programy napisane w językach skryptowych, przeznaczone do wykonywania w specjalnych środowiskach uruchomieniowych automatyzujących wykonywanie zadań.

Języki skryptowe są to najczęściej języki interpretowane, zaprojektowane z myślą o interakcji z użytkownikiem. Skrypty są też często używane do jednorazowych zadań, na przykład administracyjnych.

Środowisko programistyczne

Programowanie sprowadza się do pisania tekstu. Do tworzenia kodu można wykorzystywać wyspecjalizowane środowisko programistyczne albo zwykły notatnik tekstowy. Edytor nie jest ważny przy tworzeniu aplikacji (patrząc z punktu widzenia **kompilatora**).

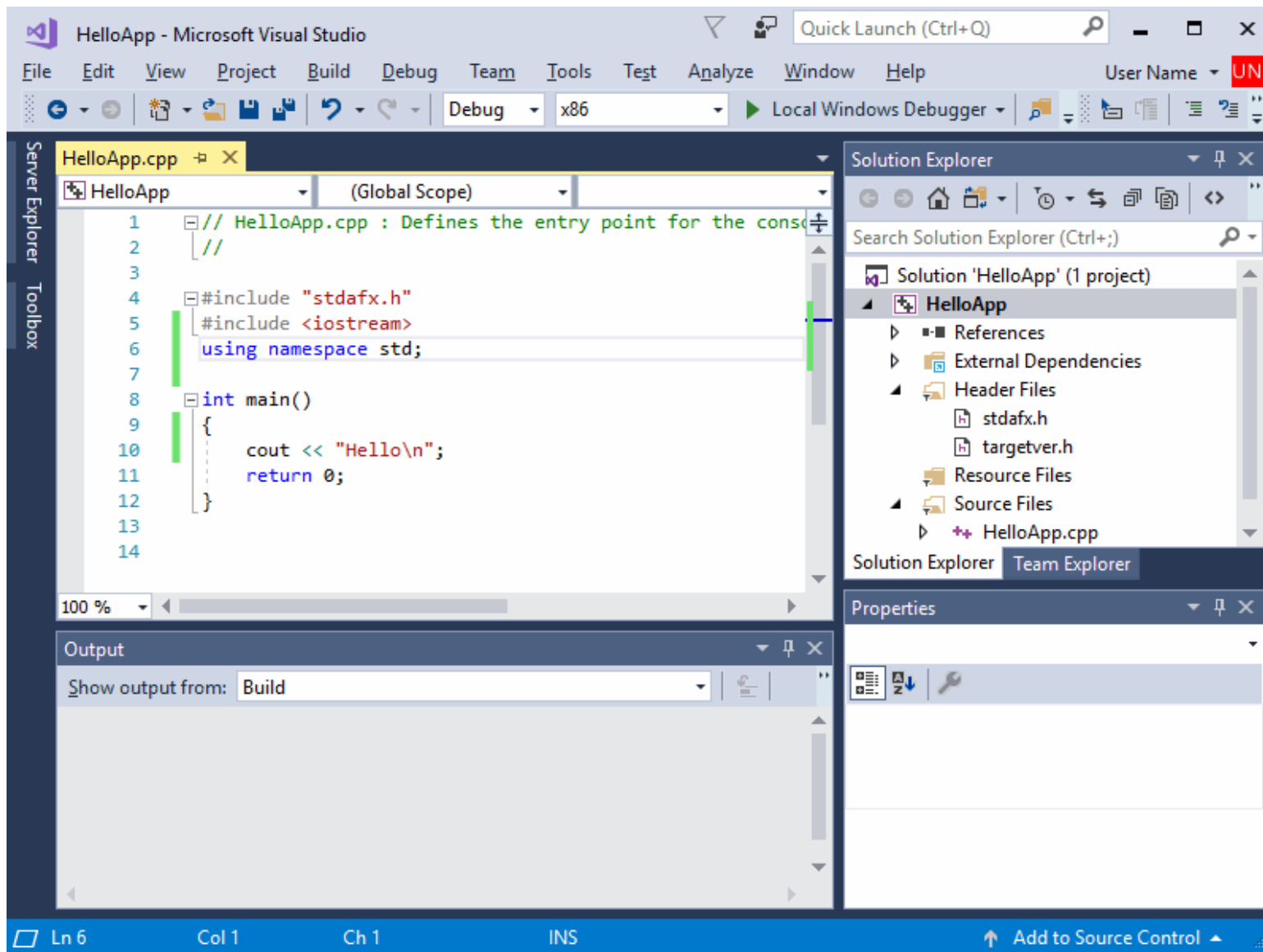
Wyspecjalizowane środowisko programistyczne zapewnia komfort pracy z językiem programowania począwszy od podstawowych aspektów takich jak kolorowanie składni czy też automatyczne wcięcia w kodzie, a skończywszy na zaawansowanych mechanizmach do **debugowania** aplikacji.

Środowisko programistyczne

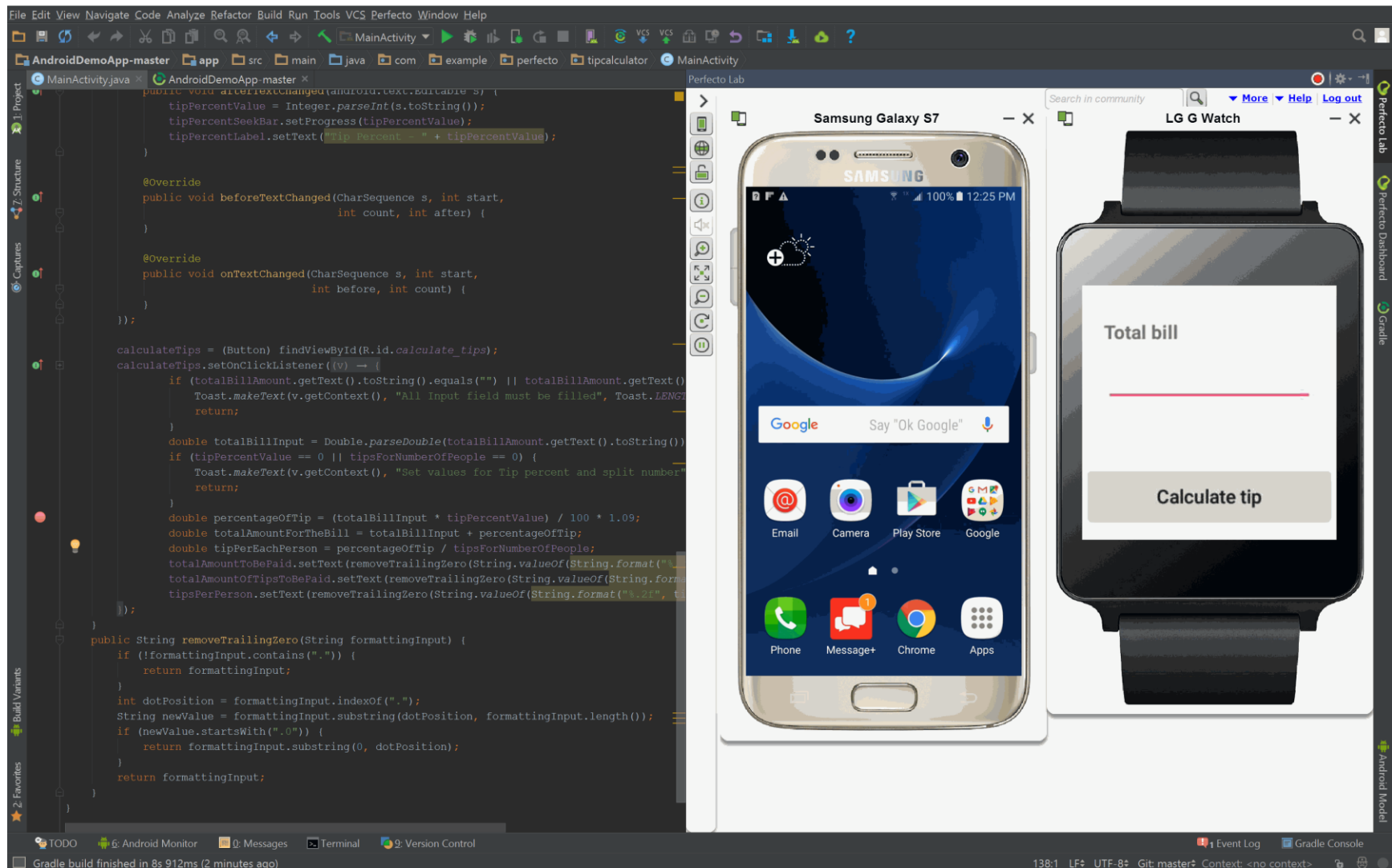
Przykładowe i jednocześnie popularne środowiska do programowania w C++: Dev-C++, wxDev-C++, Code::Blocks, Borland C++, Visual C++, Visual Studio Code

Środowisko programistyczne to aplikacja służąca do tworzenia, modyfikowania i testowania pisanych programów.

Środowisko programistyczne



Środowisko programistyczne



Budowa programu

Pierwszym ważnym zagadnieniem związanym z wytwarzaniem aplikacji jest budowa programu.

Każdy program ma swój początek i koniec.

Początek w programie jest zawsze **tylko jeden**. Nie może być ich więcej, ponieważ system operacyjny nie wiedziałby od którego miejsca może zacząć wykonywać program.

Inaczej sprawa ma się co do końca programu - może on występować w kilku różnych miejscach programu.

Budowa programu

```
#include <iostream>
```

```
/* run this program using the console pauser or add your own getch,  
system("pause") or input loop */  
using namespace std;
```

```
int main() {  
  
    cout << "Witaj swiecie" << endl;  
    return 0;  
}
```

Powyższy kod programu pomimo, iż nie jest zbyt ambitny prezentuje klasyczną budowę programów pisanych w języku C++.

Wielozadaniowość systemu, a pisanie własnych programów

Komputer jest urządzeniem pracującym w pierwotnym założeniu **sekwencyjnie**.

Wielozadaniowość to trik, który na przestrzeni lat stał się codziennością. Pisząc własne programy nie musisz jednak zastanawiać się nad sposobem działania mechanizmu wielozadaniowości - koncentrujesz się tylko i wyłącznie na pisaniu swojego programu.

System operacyjny wraz z uruchomieniem aplikacji przydzieli mu niezbędne zasoby do pracy (w tym wydzielony obszar pamięci i przydział mocy obliczeniowej procesora).

System kontroli wersji

System kontroli wersji (ang. *version/revision control system*) – oprogramowanie służące do śledzenia zmian głównie w kodzie źródłowym oraz pomocy programistom w łączeniu zmian dokonanych w plikach przez wiele osób w różnym czasie.

Systemy kontroli wersji dzielą się na:

lokalne, pozwalające na zapisanie danych jedynie na lokalnym komputerze (np. SCCS oraz RCS)

scentralizowane, oparte na architekturze klient-serwer (np. CVS, Subversion)

rozproszone, oparte na architekturze P2P (np. BitKeeper, Code Co-op, Git, svk).

System kontroli wersji

W rozwiązaniach scentralizowanych istnieje jedno centralne repozytorium, z którym wszyscy użytkownicy systemu synchronizują swoje zmiany. Rozwiązania rozproszone pozwalają na prowadzenie równoprawnych, niezależnych gałęzi, które można dowolnie synchronizować ze sobą nawzajem, np. poprzez e-mail (Code Co-op). Wybór odpowiedniego typu systemu kontroli wersji zależy od potrzeb projektu.